

NOTES

Python Memory Management & Optimization



Python Memory Management and Optimization

✓ 1. Memory Management Fundamentals

Memory management is how a programming language handles the storage and release of data in a computer's memory. When we write a program in Python, it automatically manages memory for us, which makes Python easier to use. However, understanding the basics of memory management helps us write more efficient code.

In Python:

1. Every value we use is stored in memory.
2. When we create a variable, it points to a location in memory where that data is stored.

Python uses two strategies for memory allocation:

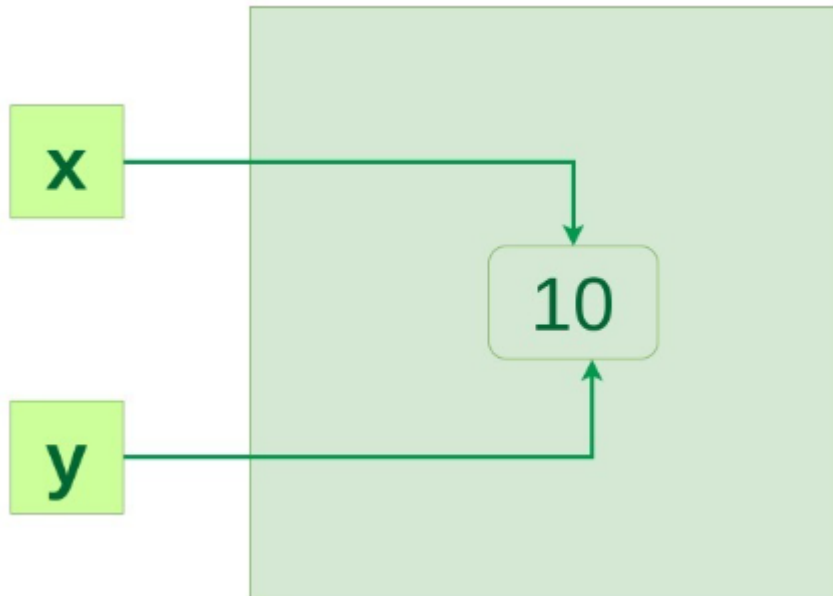
1. Reference counting
2. Garbage collection

1. Reference counting

- Reference counting works by counting the number of times an object is referenced by other objects in the system.
- When references to an object are removed, the reference count for an object is decremented.
- When the reference count becomes zero, the object is deallocated.

```
# Example :  
x = 10
```

- When `x = 10` is executed an integer object 10 is created in memory and its reference is assigned to variable `x`, this is because everything is object in Python.

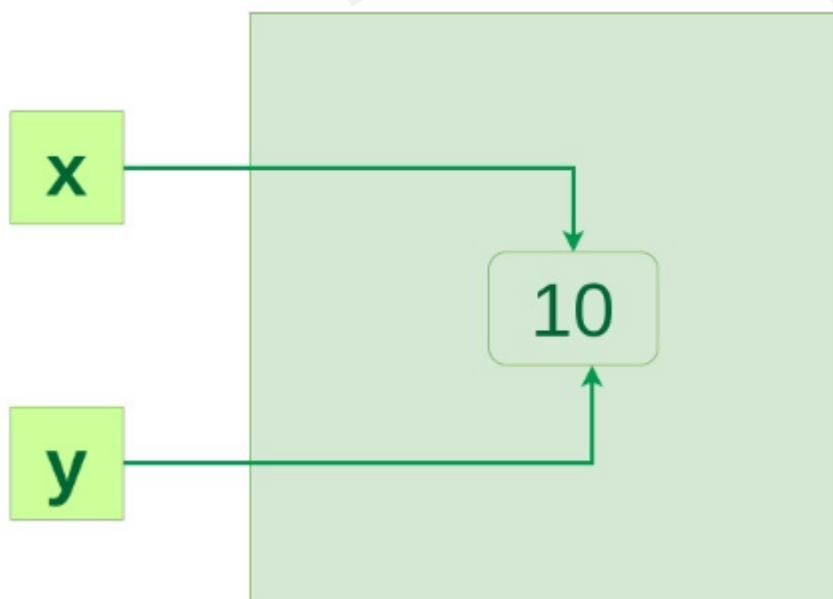


For example, Let's suppose there are two or more variables that have the same value, so, what Python virtual machine does is, rather than creating another object of the same value in the private heap, it actually makes the second variable point to that originally existing value.

```
x = 10
y = x

if id(x) == id(y):
    print("x and y refer to the same object")
```

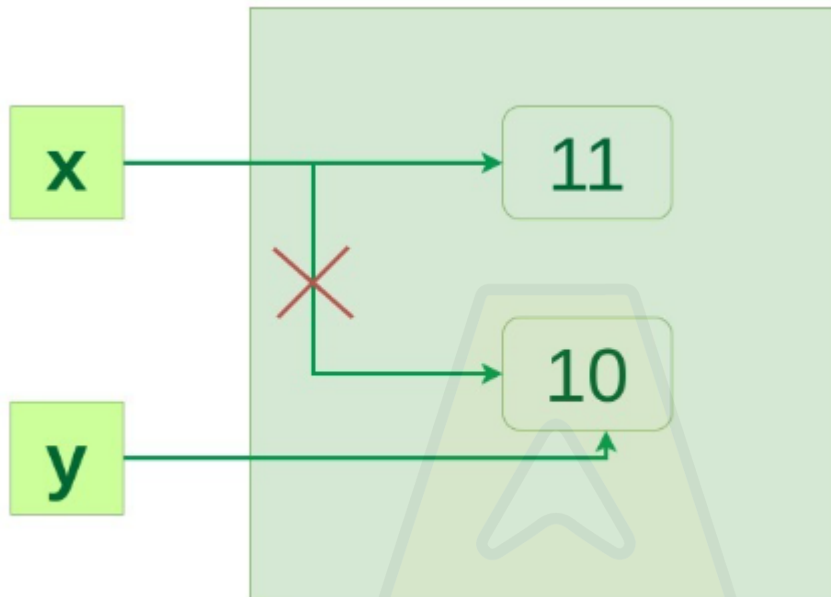
In the above example, `y = x` will create another reference variable `y` which will refer to the same object because Python optimizes memory utilization by allocation the same object reference to a new variable if the object already exists with the same value.



```
x = 10
y = x
x += 1
```

```
if id(x) != id(y):
    print("x and y do not refer to the same object")
```

So now x refer to a new object x and the link between x and 10 disconnected but y still refer to 10.



2. Garbage Collection When previously allocated memory no longer serves a purpose in your system, Python programs implement a process known as garbage collection. Garbage collection is an automatic process that detects memory the system isn't using and frees up that space. This allows you to avoid any problems with forgetting to deallocate memory. Although garbage collection is an automatic process, you do have the ability to make some modifications. For example, you can customize the time intervals in which the deallocation of memory activates.

THANK YOU



Stay updated with us!



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)